Information Sciences 740 (2026) 123216

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

A cross-layer bidirectional measurement pruning for model compression guided by feature propagation

Suyun Lian [a,1], Jiajian Cai [a,1], Yang Zhao [a,b], Stefan Poslad [c], Jihong Pei [a,*]

[a] Shenzhen University, The Guangdong Key Laboratory of Intelligent Information Processing, College of Electronics and Information Engineering, Shenzhen, 518060, China
[b] Shenzhen University, College of Computer Science and Software Engineering, Shenzhen, 518060, China
[c] Queen Mary University of London, School of Electronic Engineering and Computer Science, London, E1 4NS, UK

ARTICLE INFO

Keywords:
Pruning
Significant kernel
Cross-layer
Pruning rate
Batch normalization

ABSTRACT

The substantial computational cost and large number of parameters in deep convolutional neural networks (DCNNs) restrict their deployment on resource-constrained edge devices. Channel pruning offers a practical solution to this problem. However, most existing pruning methods assess channel importance solely within individual layers, neglecting how information propagates across layers. To overcome this limitation, we propose a cross-layer bidirectional measurement strategy that jointly considers both the synthesis perspective and the distribution perspective of each channel. This strategy enables a more comprehensive evaluation of structural importance. To improve measurement accuracy, we focus on significant kernels, thereby reducing the noise introduced by low informative or utility kernels. Furthermore, to mitigate this issue, we introduce an adaptive pruning rate allocation strategy based on the global statistics of batch normalization (BN) parameters. During the fine-tuning stage, we apply knowledge distillation, using the original model as a teacher to guide the pruned student model. Extensive experiments on multiple datasets and architectures validate the effectiveness of the proposed cross-layer bidirectional measurement(CLBM) framework. On the VGG16-CIFAR10 benchmark, CLBM achieves an acceleration of 13.38× in terms of parameters, and an acceleration of 5.27× in floating-point operations (FLOPs), while maintaining competitive accuracy with only a 0.17% drop in classification performance.

1. Introduction

With the rapid advancement of data availability and computational resources in artificial intelligence, deep learning models have evolved toward increasing architectural complexity [1]. To pursue higher accuracy, better generalization, and the capability to tackle complex tasks, modern models often contain hundreds of millions, billions, or even trillions of parameters [2]. Both the depth (i.e., number of layers) and width (i.e., number of channels) of these models are expanding significantly [3]. However, such complexity demands enormous computational resources, such as floating point operations(FLOPs), during training and inference. This reliance on costly GPU/TPU clusters consumes substantial storage capacity. In practice, such as in autonomous driving [4], mobile device vision [5], and medical diagnostics [6], where edge computing is commonly deployed hardware constraints prohibit the deployment of these massive models. Model compression techniques are essential to bridge the gap between model complexity and deployment feasibility. The goal of model compression is to significantly reduce model size and computational demands while preserving accuracy.

* Corresponding author.
Email address: jhpei@szu.edu.cn (J. Pei).
[1] Suyun Lian and Jiajian Cai contributed equally to this work.

https://doi.org/10.1016/j.ins.2026.123216
Received 11 August 2025; Received in revised form 3 February 2026; Accepted 6 February 2026
Available online 7 February 2026
0020-0255/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Mainstream compression approaches include quantization [7], knowledge distillation [8], low rank decomposition [9], and pruning [10]. Quantization reduces the numerical precision of weights and activations (e.g., from 32-bit float to 8-bit integer). Knowledge distillation enables a compact student model to learn from a larger teacher model. Low rank decomposition factorizes large matrices into smaller components. Pruning eliminates redundant or unimportant components from the network. Among these, pruning, particularly structured pruning, offers notable advantages due to its flexibility, generalization, and potential to uncover the internal workings of neural networks [11].

The core principle of pruning is to remove weights, channels, or modules that minimally impact performance [12]. Two critical challenges in pruning research are: (1) accurately identifying redundant structures and assessing their importance; and (2) designing optimal pruning rates to retain essential network components and support performance recovery. Existing channel pruning methods predominantly evaluate importance within individual layers, relying on criteria such as filter norms or gradient magnitudes [10,13]. However, the local perspective neglects the inter layer dependency of features and fails to capture how channel information propagates and contributes to downstream representations. In deep convolutional neural networks (DCNNs), distinct convolution kernels within the same filter exhibit significant variations in their capacity for information transmission. Some become inefficient due to sparse weights or low activation strength. Aggregating all kernels for evaluation risks misjudging the utility of filters. As a result, current approaches often misjudge structural redundancy, leading to inconsistent pruning decisions and suboptimal recovery.

To bridge these limitations, we propose a cross-layer bidirectional measurement (CLBM) that jointly models the synthesis capability of a layer to generate features and the distribution of these features in subsequent layers. By focusing on significant convolution kernels, CLBM isolates high information substructures, providing a refined foundation for channel evaluation. Furthermore, unlike conventional approaches that require manual layer wise pruning rate specification, CLBM constructs pruning sensitivity from all batch normalization (BN) parameters to adaptively allocate the global pruning budget, while employing self distillation from the original model to preserve discriminative knowledge. This unified design ensures that pruning and fine-tuning are coherently aligned, enhancing both efficiency and generalization. Through this design, CLBM bridges the long standing gap between redundancy identification and knowledge recovery, yielding a compact yet high performing network.

Our contributions are summarized as follows:

- We propose a cross-layer bidirectional metric. Our method combines the perspectives of channel synthesis and distribution, and considers the essential characteristics of feature transmission to design a new structural importance quantification method, which makes our structural redundancy metric more comprehensive and accurate.
- We propose using significant convolution kernels to measure structural importance. Significant convolution kernels reduce interference caused by low information convolution kernels, providing a more refined foundation for quantifying structural importance.
- We propose a new adaptive pruning rate setting method. By constructing layer pruning sensitivity through BN layer parameters, we allocate pruning rates for each layer, reducing the subjectivity of manual settings. Moreover, it integrates self distillation from the original model, ensuring coherent alignment between pruning and fine-tuning, thus enhancing efficiency and generalization.

2. Related work

The measurement of the importance of channels or filters is one of the key components in pruning methods. The most common strategy is to measure importance based on the magnitude of weights. Li et al. [14] proposed using the L_1 norm of filters as an importance metric, arguing that filters with smaller norms have a lesser impact on model performance and therefore could be prioritized for pruning; He et al. [15] set pruning probabilities based on the L_2 norm, gradually removing low importance filters using soft pruning; Zhu et al. [16] further introduced a mixed norm design, evaluating importance at three scales—filter, group, and layer—and retaining or pruning convolution kernels based on sparsity indices. Back [17] proposed a dynamic sparse model structure using the magnitude based method with attention. As research has progressed, pruning methods that rely solely on the magnitude of norms have increasingly exposed their limitations in terms of robustness and generalization ability. Moreover, such methods overlook the structural redundancy between convolution kernels. To address this, some studies have begun to analyze convolution kernels from a similarity perspective. He et al. [18] proposed the FPGM method, arguing that geometric median ratios better reflect the representativeness of filters than norms, and compressed the model by identifying and pruning redundant filters; Pei et al. [19] constructed a filter similarity graph based on the mapping capability of convolution kernels within filters and the similarity between equivalent convolution kernels, and used graph partitioning methods for clustering and pruning. Chang et al. [20] identified structural redundancy by dual side clustering of filters and feature maps. Li et al. [21] clustered kernels into distinct categories, considering kernels within the same category as redundant. In addition to the above strategies, several new importance metrics have emerged in recent years, such as pruning methods based on feature map rank [22], and pruning methods based on attention distribution [23].

Although numerous studies have proposed a variety of pruning metrics, they commonly suffer from a fundamental limitation: the importance of channels is evaluated solely based on static information from the current layer, without accounting for the dynamic role of channels in information transmission across layers during forward propagation. In practice, some channels that exhibit strong activations in the current layer may generate features that are poorly utilized by subsequent layers. This results in a supply–demand mismatch, where high supply (i.e., strong activations) does not correspond to actual demand (i.e., downstream usage). Such evaluation methods overlook the efficiency of information propagation. This leads to the erroneous pruning of critical channels and ultimately degrades model performance. Therefore, developing cross-layer channel importance metrics that incorporate both upstream synthesis and downstream distribution is a pressing challenge and a key research direction in the field of network pruning.

The design of pruning rates plays a crucial role in determining the performance of pruning. Pruning rates dictate the number of channels (or convolution kernels) to be removed in each layer, and their configuration directly influences the final network architecture. The most commonly adopted approach is to apply predefined pruning rates, in which uniform or layer specific pruning rates are manually set. For instance, in some studies [18,24], a fixed pruning rate was applied across all layers, offering simplicity and ease of implementation. However, due to substantial variations in redundancy and pruning sensitivity across layers, such uniform strategies failed to produce an optimal sub-network structure. An alternative approach is global threshold based pruning [25,26], where a single threshold was applied to channel importance scores to determine the pruning extent in each layer. Although this method simplified pruning rate selection, it was unable to account for variations in scale and distribution of importance scores across layers, leading to unbalanced pruning in shallow or deep layers and degrading overall performance. Another line of research explored learnable masking mechanisms [27], which introduced a trainable mask for each channel to identify redundant components during joint optimization. Recent works leveraged intelligent optimization techniques. For example, Dong et al. [28] employed a genetic algorithm to allocate pruning rates across layers, given a specified overall pruning rate. Wei et al. [29] incorporated neural architecture search methods to identify high performance pruning rates, thereby obtaining optimal pruning structures. Wang et al. [30] formulated pruning as a reinforcement learning problem, where an agent autonomously determines layer wise pruning rates to balance compression and accuracy. Liu et al. [31] leveraged a meta learning based super-net to generate pruned sub-networks with different pruning rates. While these approaches improved automation and flexibility, they also suffered from high computational complexity and poor convergence due to the direct search over the entire pruning configuration space.

Existing pruning rate design methods still suffer from shortcomings such as heavy reliance on manual settings, high training costs, and insufficient fairness between layers. There is a clear need for a new strategy that can automatically sense the pruning sensitivity of each layer and reasonably allocate the pruning rate to obtain more efficient and superior pruning results.

3. Background

3.1. Signals propagate in filter and group

In DCNNs, convolution kernels determine not only how features are extracted but also how signals propagate between channels. Fig. 1 illustrates the signal flow between the l-th and $(l+1)$-th convolution layers. A filter refers to a set of kernels that transmit information from multiple input channels to a single output channel. In Fig. 1, each row of kernels F_n^l represents a filter, functioning as a feature synthesizer that aggregates input signals to construct new features. In contrast, a group comprises kernels that transmit information from a single input channel to multiple output channels. Each column G_m^l represents a group, acting as a feature distributor that propagates the same input to various outputs. Feature transmission occurs via inter layer channel connections, where the output channel Y^l of layer l serves as the input X^{l+1} to layer $l+1$. As a result, each intermediate channel fulfills two critical roles: feature synthesis and feature distribution. As a synthesizer, Y^l is generated from X^l by the convolution kernels of layer l; as a distributor, X^{l+1} transmits information to the next layer for further feature extraction.

The synthesizer refers to the role of aggregating information from the input channels of the current layer via its convolution filters to produce the output channel Y^l. In contrast, the distribution denotes the use of X^{l+1} as the input for the next layer, where it undergoes feature extraction through the next layer's convolution filters and is distributed to multiple output channels. From the synthesizer's perspective, convolution kernels are critical in shaping the type and richness of extracted features. For instance, in image recognition tasks, kernels of varying sizes and weights can effectively capture essential information such as edge contours and texture patterns across multiple scales. However, evaluation strategies that focus only on this perspective assess channel importance based on the synthesis capacity of the current layer, but do not consider how the resulting features propagate through the network, which may fail to capture the true significance of a channel. From the distribution's perspective, the downstream layer's capacity to receive and utilize input features is a vital factor in the model's overall performance. Each layer selects, integrates, and further abstracts the received signals according to its specific task and architectural characteristics. In deep networks, later layers often require more abstract representations to support complex tasks such as classification or detection.

The output channels of the current layer also serve as input to the next layer. Their informativeness is reflected by the activation response of the corresponding convolution kernel groups in the subsequent layer. If a group exhibits weak activation, such as those

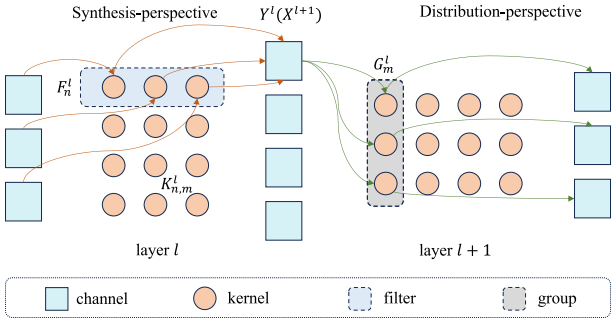


Fig. 1. Illustration of signal propagation through filters and groups.

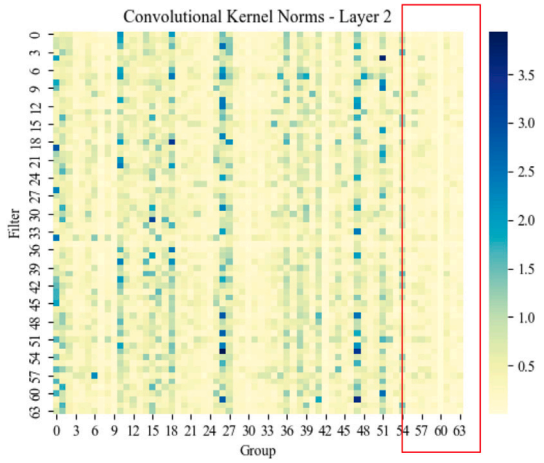


Fig. 2. An example of poor group.

highlighted in the red box in Fig. 2, this pattern suggests limited feature extraction capability. This implies that the corresponding input channel (i.e., the output from the previous layer) is of low importance. Therefore, evaluating channel importance solely from the synthesis perspective is insufficient. A comprehensive assessment should also incorporate the distribution's perspective to fully capture the role of intermediate channels in feature transmission.

3.2. Batch normalization

Batch Normalization (BN) layer is a prevalent technique that aims to expedite the training process and enhance model performance. BN layers standardize the input data for each layer, thereby enhancing data stability and mitigating the issues of vanishing and exploding gradients in deep neural network training. The calculation process is as follows:

$$Z_{norm} = \frac{Z_{in} - \mu_B}{\sqrt{{\sigma_B}^2 + \epsilon}}, \tag{1}$$

$$Z_{out} = \gamma Z_{norm} + \beta, \tag{2}$$

where Z_{in} denotes the input data that has undergone processing via the convolution layer. The parameters μ_B and ${\sigma_B}^2$ represent the mean and variance of a mini batch, respectively. The quantity ϵ is of negligible magnitude, serving the purpose of ensuring that the denominator does not become zero. By standardizing Z_{in}, Z_{norm} with a mean of 0 and a standard deviation of 1 is obtained. To ensure the representational capacity of the model, it introduces two learnable parameters, the scaling factor γ and the bias factor β, to restore the expressive power of the network. As demonstrated in Eq. (2), this adjustment effectively calibrates Z_{norm} to yield an output Z_{out} with a mean of β and a standard deviation of γ.

The BN layer is generally situated between the convolution layer and the activation function. In summary, the output of the BN layer determines the input of the subsequent layer. If the value of the output channel subsequent to the BN layer is less than 0, then the subsequent ReLU activation function sets it to 0. As a result, the output information is lost. After processing through the BN layer, if most of the activation values of Z_{out} are less than 0 or close to 0, it indicates that the current convolution layer is less sensitive to pruning. It has been established that each filter corresponds to a set of scaling factors denoted as γ, and bias factors denoted as β, which determine the values of the output channels passing through that filter. Therefore, these parameters can be used to guide the sensitivity of the convolution layer to filter pruning.

4. Methodology

In this section, we present the CLBM framework designed for DCNNs. The overall pipeline of CLBM is illustrated in Fig. 3. For a given pretrained model, CLBM first analyzes channel importance by leveraging a bidirectional metric that integrates both the synthesis perspective (current layer generation) and the distribution perspective (feature usage in the next layer). To ensure accuracy, the analysis focuses on significant convolution kernels, filtering out noisy or low efficiency kernels. Based on the importance scores, CLBM computes a global pruning sensitivity profile using BN statistics, and then automatically assigns adaptive pruning rates to layers. After pruning, the network is fine-tuned via knowledge distillation, using the original model as a teacher to guide the recovery of the pruned student model. This pruning–retraining process yields a compact, highly efficient network ready for deployment under resource constrained scenarios.

In the following sections, Section 4.1 analyzes the distribution patterns of weights, focusing primarily on the distribution patterns of convolution kernel weights and BN parameters. Section 4.3 provides a detailed introduction to the specifics of the bidirectional metric. Section 4.4 details the specifics of adaptive pruning rates. Section 4.5 introduces the specifics of knowledge distillation fine-tuning, and finally, the entire framework is summarized in pseudo-code.

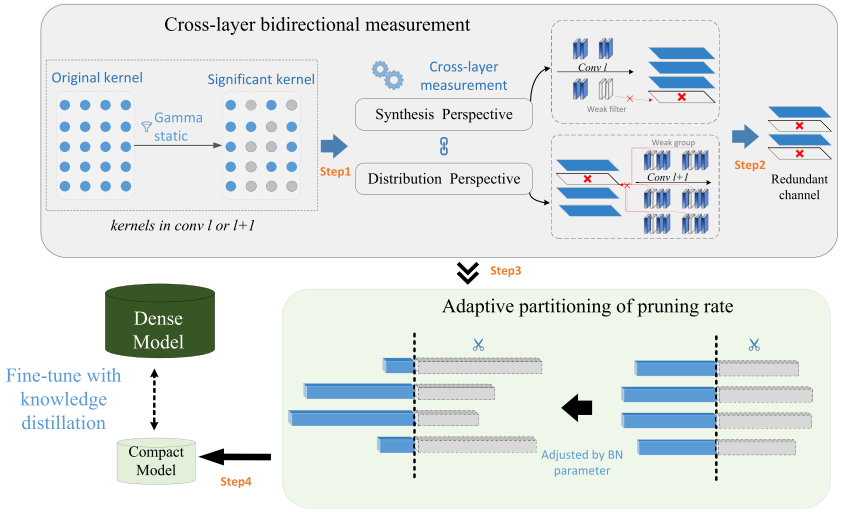


Fig. 3. The framework of CLBM algorithm. Step 1: Significant kernels are selected via gamma distribution thresholding to generate a binary mask $m \in \{0,1\}$, as Eq. (8); Step 2: Channel importance is computed using a bidirectional metric where the synthesis and distribution perspectives jointly determine the score $SG_n^l \cdot SR_n^l$, as Eq. (11); Step 3: Pruning rates are adaptively assigned based on BN statistics using γ and β, as Eq. (12); Step 4: The pruned network is fine-tuned via knowledge distillation by loss.

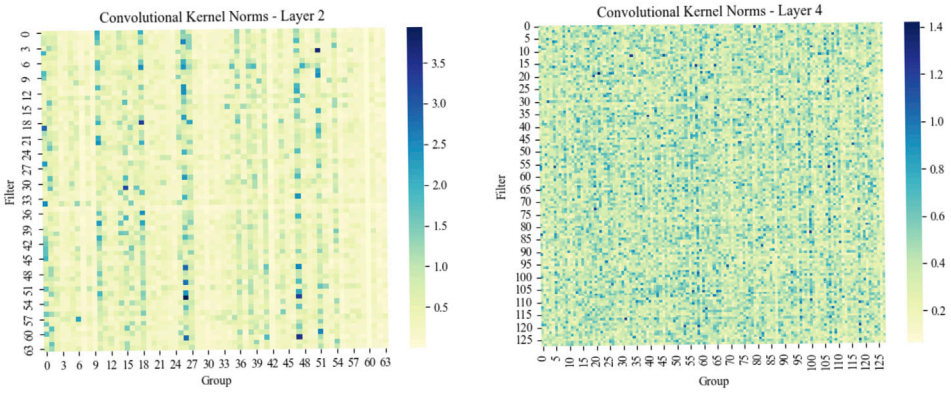


Fig. 4. Heatmap of L_1 norms of kernels in the second and fourth layers of VGG16.

4.1. Weight distribution

4.1.1. Kernel weight distribution

Before defining precise criteria for evaluating the convolution kernels importance, it is essential to perform a statistical analysis of their weight distributions. Fig. 4 presents a visualization of the L_1 norms of convolution kernels in the second and fourth layers of VGG16. Fig. 4 reveals the existence of numerous kernels with low information transmission efficiency. These kernels, referred to as insignificant convolution kernels, have little impact on the quality of the corresponding output channels. Including such kernels in the evaluation process can introduce noise and hinder the accurate assessment of filter or channel importance.

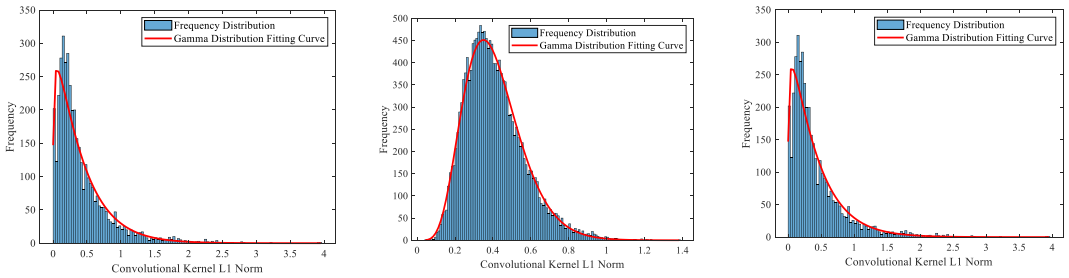
Fig. 5 further illustrates the statistical distribution of the L_1 norms of kernels in the 2nd, 4th, and 8th layers of VGG16. As observed, these distributions resemble a gamma distribution, characterized by two parameters: the shape parameter k and the scale parameter θ, denoted as $\Gamma(k,\theta)$. The probability density function (PDF) of the gamma distribution is expressed as:

$$f(x;k,\theta) = \frac{x^{k-1}e^{-x/\theta}}{\Gamma(k)\theta^k}, \tag{3}$$

where x denotes the random variable, and $\Gamma(k)$ is the gamma function, which is defined as:

$$\Gamma(x) = \int_0^{+\infty} t^{x-1}e^{-t}dt, \quad s.t. \quad x > 0. \tag{4}$$

As illustrated in the Fig. 5, the data demonstrate a pronounced long tail distribution. The L_1 norm values across all layers are concentrated within a narrow range, with a long right side tail, indicating that most convolution kernels have small norms, while only a few exhibit large values. Moreover, the shape parameters of the Γ distributions differ significantly across convolution layers. The second layer's distribution peaks near zero, suggesting a high proportion of redundant convolution kernels with weak responses.



(a) Statistic of 2nd layer. (b) Statistic of 4th layer. (c) Statistic of 8th layer.

Fig. 5. Statistical distribution of different layers in VGG16.

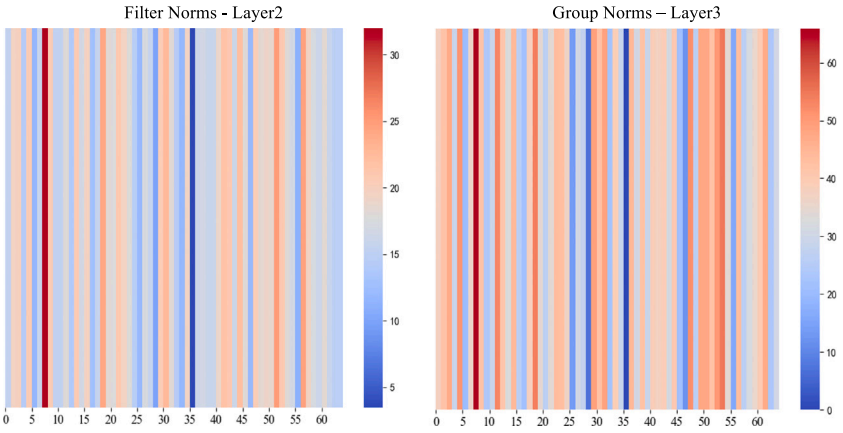


Fig. 6. Weights of filters and groups of 2nd and 3rd layers in VGG16.

In contrast, the peak positions of the fourth and eighth layers shift rightward, implying stronger kernel responses, fewer redundant kernels, and clearer differentiation in kernel importance.

Although all three distributions approximate Gamma distributions, their shape and scale parameters vary notably. Traditional pruning methods often apply a uniform threshold—such as top-k selection or a fixed norm value—across all layers. However, this approach fails to account for the heterogeneity in response distributions among layers, overlooking intra layer structural nuances and potentially leading to the removal of important convolution kernels.

To intuitively understand how filters and groups contribute to information transmission, and to examine their necessity from the perspectives of feature synthesis and distribution, we visualize and analyze the weights of filters and groups. As shown in Fig. 6, the norm values of the filters in the second layer (current layer) and the groups in the third layer (next layer) of VGG16 are displayed. These values represent the importance metrics from the synthesis perspective and the distribution perspective for each channel, respectively. In the left part of Fig. 6, larger norm values indicate stronger feature extraction capabilities of the filters, suggesting higher importance of the corresponding output channels from the synthesis perspective. In the right part of Fig. 6, larger norm values reflect stronger feature processing capabilities of the groups, indicating higher importance of the input channels from the distribution perspective. As illustrated, the importance metrics of the synthesis and distribution perspectives are not always positively correlated. For instance, the 11th channel has a medium level importance metric on the synthesis perspective but a relatively high importance on the distribution perspective, indicating effective information transmission to the next layer. Conversely, the 15th channel exhibits a relatively high synthesis perspective importance but a low distribution perspective value, suggesting limited information propagation despite good feature synthesis. Similar discrepancies are observed for channels such as the 45th and 61st. These observations are not limited to the examined layers; other convolution layers in the network also exhibit a lack of positive correlation between synthesis perspective and distribution perspective importance metrics, highlighting the necessity of evaluating both aspects jointly when assessing channel significance.

4.1.2. BN weight distribution

In BN layers, channel importance evaluation based solely on the scaling factor γ yields satisfactory results when γ and the shift factor β are positively correlated. Conversely, this approach becomes unreliable when such correlation is absent. To examine the relationship between γ and β, we visualize BN layer parameters across lower, middle, and upper layers of VGG16. Fig. 7 presents the parameter distributions for VGG16's second convolution layer. Consider the second layer of VGG16 (Fig. 7). Group 39 exhibits the smallest β value in this layer, but its corresponding γ is not minimal, ranking in the upper-middle range. Conversely, group 30 has the largest β, but its γ is only mid range. Similarly, group 22 shows the maximum γ while its β is merely average. These observations

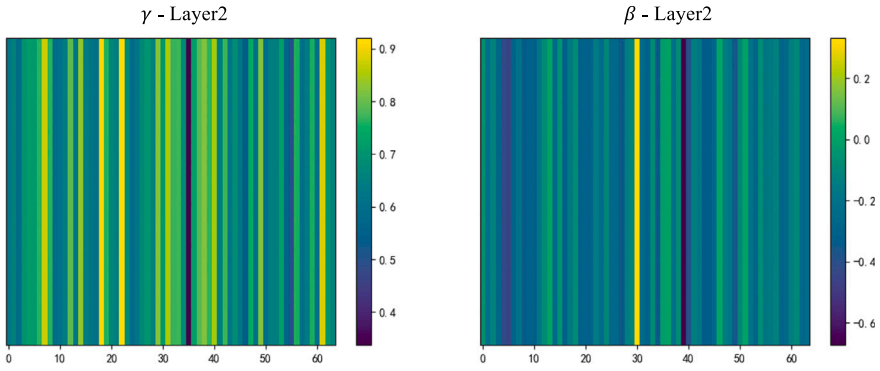


Fig. 7. Visualization of BN layer parameters in the second layer of VGG16.

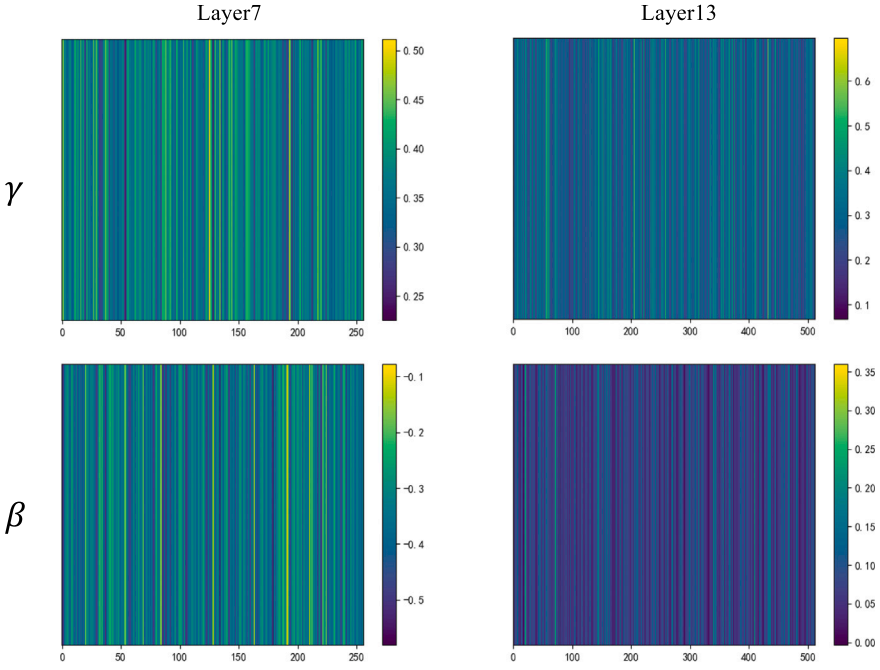


Fig. 8. Visualization of BN layer parameters in the 7th and 13th layer of VGG16.

demonstrate an absence of positive correlation between γ and β within BN parameter groups. Beyond lower level BN layers, we further visualize parameters from middle and upper layers (Fig. 8). This comprehensive analysis reveals that large γ values coexist with both high and low β values, and vice versa. Effective evaluation based solely on γ requires positive correlation between γ and β, which is rarely observed in practical neural network BN layers.

4.2. Preliminaries

Assume that a DCNN has L convolution layers, and the l-th convolution layer ($1 \le l \le L$) has M_l input channels and N_l output channels. Generally, the set of convolution kernels applied to produce a single output channel is referred to as a "filter", whereas the set of kernels corresponding to a single input channel is typically called a "group", as shown in Fig. 1. When representing the parameters of a convolution layer by using filters, the weights W^l of the l-th convolution layer are denoted as $W^l = \{F_1^l, \dots, F_n^l, \dots, F_{N_l}^l\} \in \mathbb{R}^{N_l \times M_l \times H_l \times W_l}$, where F_n^l denotes the n-th filter of layer l. Each filter can be further represented as $F_n^l = \{K_{n,1}^l, \dots, K_{n,m}^l, \dots, K_{n,M_l}^l\} \in \mathbb{R}^{M_l \times H_l \times W_l}$, where $K_{n,m}^l$ denotes the m-th kernel of the n-th filter in the l-th layer. Alternatively, when using groups to describe these parameters, the same weights W^l can be expressed as $W^l = \{G_1^l, \dots, G_m^l, \dots, G_{M_l}^l\} \in \mathbb{R}^{M_l \times N_l \times H_l \times W_l}$, where G_m^l represents the m-th group of the l-th layer. Each group can be further defined as $G_m^l = \{K_{1,m}^l, \dots, K_{n,m}^l, \dots, K_{N_l,m}^l\} \in \mathbb{R}^{N_l \times H_l \times W_l}$. Although the notation $K_{n,m}^l$ appears in both representations, it conveys slightly different meanings. In the filter based view, it describes how kernels are organized per output channel, while in the group based view, it emphasizes organization per input channel. These perspectives provide complementary insights into the parameter structure of convolution layers. The output features of the l-th convolution layer

are denoted as $Y^l = \{Y_1^l, Y_2^l, \ldots, Y_n^l, \ldots, Y_{N_l}^l\} \in \mathbb{R}^{N_l \times H_Y^l \times W_Y^l}$, and the input features are denoted as $X^l = \{X_1^l, X_2^l, \ldots, X_m^l, \ldots, X_{M_l}^l\} \in \mathbb{R}^{M_l \times H_X^l \times W_X^l}$. The computational relationship between the inputs and outputs is given by $Y^l = \sigma(X^l \circledast W^l)$, where $\circledast$ denotes the convolution operation and $\sigma(\cdot)$ is a nonlinear activation function.

4.3. Cross-layer bidirectional measurement

Existing pruning methods often employ uniform thresholds, neglecting variations in activation distributions across the convolution layers. This oversight risks pruning critical channels. To accurately evaluate convolution kernel importance, our method leverages gamma distributed kernel norm statistics. We derive layer adaptive pruning thresholds from distribution parameters, enhancing pruning accuracy and robustness. The mean and variance of the Gamma distribution are given by:

$$E[X] = k\theta, \tag{5}$$

$$Var(X) = k\theta^2. \tag{6}$$

After obtaining the mean E_l and variance V_l of the L_1-norm distribution for convolution kernels in layer l, we derive layer specific thresholds for significant kernel selection by the formula:

$$T_l = E_l - c\sqrt{V_l}, \tag{7}$$

where T_l denotes the selection threshold for layer l, and c is an adjustable parameter controlling pruning intensity. This formulation enables flexible threshold adaptation across diverse network architectures and tasks. Using these layer adaptive thresholds, we construct a binary indicator tensor D that identifies significant kernels:

$$D_{n,m}^l = \begin{cases} 1 & ||K_{n,m}^l||_1 \geq T_l \\ 0 & ||K_{n,m}^l||_1 < T_l \end{cases}, \tag{8}$$

where $||K_{n,m}^l||_1$ represents the L_1 norm of the m-th kernel in the n-th filter of layer l. Kernels with norms meeting or exceeding their layer threshold (T_l) are preserved (marked as 1), while sub threshold kernels are pruned (marked as 0). Taking the second and fourth convolution layers of VGG16 as examples, the distribution of significant convolution kernels is illustrated in Fig. 9. As shown in Fig. 9, the proposed method effectively distinguishes between significant and insignificant convolution kernels within each layer.

Existing channel pruning methods predominantly evaluate channel importance solely from the synthesis perspective—retaining channels based on filter level significance metrics. This approach implicitly assumes that channels making substantial contributions at their native layer maintain value throughout the network. However, this single perspective evaluation suffers from significant limitations, as it disregards downstream feature distribution, frequently preserving strong but redundant channels. Our method extends importance assessment to a cross-layer paradigm, jointly considering the synthesis perspective and distribution perspective. This dual aspect framework operates as a supply demand matching mechanism, simultaneously assessing feature supply and demand of downstream.

Consequently, after identifying significant convolution kernels for each layer, our method defines two importance metrics per channel from the synthesis and distribution perspectives. The synthesis metric is computed for each output channel's convolution kernels in the current layer. Thus, after applying significant kernels, the synthesis importance for channel i is given by:

$$SG_n^l = \sum_{m=1}^{M_l} ||K_{n,m}^l||_1 \cdot D_{n,m}^l, \tag{9}$$

where SG_n^l denotes the synthesis perspective metric for the n-th output channel in the l-th convolution layer. $||K_{n,m}^l||_1$ represents the L_1 norm of the m-th convolution kernel within the n-th filter of the l-th convolution layer, and M_l denotes the number of input channels to the l-th convolution layer. The binary indicator $D_{n,m}^l$ signifies whether the m-th kernel in the n-th filter of the l-th convolution layer

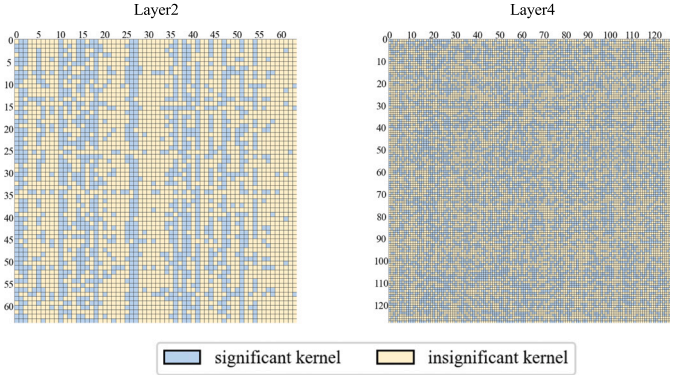


Fig. 9. Significant convolution kernel distribution of 2nd and 4th layers in VGG16.

is considered significant. The value of SG_n^l reflects the generative quality of the corresponding output channel in the current layer. A larger SG_n^l indicates higher importance from the perspective of output contribution. When SG_n^l reaches its maximum, the n-th output channel exhibits the highest generative capacity among all channels in the l-th convolution layer.

In addition to evaluating the importance of a channel from the synthesis perspective, it is also essential to define an importance metric from the distribution perspective. The distribution perspective importance is computed for each group of input channel convolution kernels in the subsequent layer. Specifically, each output channel in the current layer corresponds to a group of input channel kernels in the next convolution layer. Hence, incorporating only the significant convolution kernels, the distribution perspective importance metric for each channel is defined as follows:

$$SR_n^l = \sum_{i=1}^{N_{l+1}} ||K_{i,n}^{l+1}||_1 \cdot D_{i,n}^{l+1}, \tag{10}$$

where SR_n^l denotes the distribution perspective importance of the n-th output channel in the l-th convolution layer. $||K_{i,n}^{l+1}||_1$ denotes the L_1 norm of the i-th convolution kernel in the n-th group of the $(l+1)$-th convolution layer, and N_{l+1} denotes the number of output channels in the $(l+1)$-th layer. The binary indicator $D_{i,n}^{l+1}$ determines whether the i-th kernel in the n-th group is significant. The metric SR_n^l reflects the channel's distribution quality from the perspective of the next layer. A higher value of SR_n^l implies greater significance in terms of downstream signal propagation. When SR_n^l reaches its maximum, it indicates that the n-th output channel in the current layer possesses the strongest signal transmission capability.

After computing the importance metrics from both the synthesis and distribution perspectives, the two values are integrated to assess the overall importance of each channel. For all convolution layers except the final one, both the input side (synthesis) and the output side (distribution) importance metrics are available. For the output channels in the last layer where no subsequent group convolution kernels exist, so the synthesis importance metric is used exclusively. The combined importance metric for each channel is defined as follows:

$$SV_n^l = \begin{cases} SG_n^l \cdot SR_n^l, & \text{if } l \neq L \\ SG_n^l, & \text{if } l = L \end{cases}, \tag{11}$$

where L denotes the total number of layers in the network, and SV_n^l represents the unified importance score for the n-th output channel in the l-th convolution layer. For non terminal layers, this value is obtained by multiplying the synthesis perspective importance metric SG_n^l and the distribution perspective importance metric SR_n^l. The proposed bidirectional measurement is computed only once during the offline pruning stage, therefore adding only minimal processing cost. The synthesis perspective metric reflects the capacity of the current layer's convolution kernel to extract features from the input and generate informative output channels. In contrast, the distribution perspective metric evaluates the ability of the subsequent layer to receive and utilize information transmitted from the current layer. Their product yields a comprehensive evaluation of each channel's overall contribution, considering both feature synthesis and signal propagation.

For instance, a channel with a high synthesis perspective importance but low distribution perspective importance suggests effective feature extraction but poor transmission capability, leading to a limited overall contribution. Conversely, only when both metrics are high does a channel play a critical role in the network. Compared with traditional approaches that consider only a single perspective, this dual metric evaluation framework provides a more balanced and informative assessment. It enables the identification of channels that are strong in one aspect but weak in the other, thus supporting more targeted and effective pruning strategies. Ultimately, this method helps streamline the model structure and enhance computational efficiency without sacrificing performance.

4.4. Adaptive partitioning of pruning rate

Numerous studies have demonstrated that the sensitivity of different convolution layers in DCNNs to pruning varies significantly. Over pruning highly sensitive layers may lead to an irreversible degradation in model accuracy that cannot be recovered through fine-tuning. Therefore, it is essential to assign appropriate pruning rates to each convolution layer. In many existing methods, pruning rates are manually specified for individual layers, introducing a high degree of subjectivity. To address this, the proposed method establishes a correlation between the layer wise pruning sensitivity and pruning rate by leveraging the scaling factor γ and bias factor β from the BN layer. These parameters are used to construct the pruning sensitivity evaluation for each layer. Based on this evaluation and a predefined overall pruning ratio, the pruning rate for each layer is automatically determined.

In prior approaches, the scaling factor γ is commonly used as the sole indicator for modeling channel importance or pruning sensitivity. These methods typically assume that γ directly influences the magnitude of the output channels, with each γ corresponding to a specific filter. Under this assumption, a convolution layer containing many γ values close to zero implies the presence of redundant filters that can be safely removed. However, in practice, the output value of a channel does not depend exclusively on γ. According to the computation process of the BN layer, the normalized output Z_{out} is influenced by both the scaling factor γ and the bias factor β, as demonstrated in Section 4.1.2. Therefore, incorporating both factors yields a more accurate estimation of channel sensitivity. Our method combines the scaling factor γ and the bias factor β to construct a pruning sensitivity measure for each convolution layer. In order to enhance the distinction between critical channels (channels whose BN scaling factors exhibit extreme magnitudes), we applied a square root transformation to γ, which avoids the erroneous removal of critical channels. Specifically, the pruning sensitivity

score is computed for each pair of BN parameters associated with the convolution layer using the following formula:

$$I_i^l = \mu\sqrt{|\gamma_i^l|} + (1-\mu)\beta_i^l, \tag{12}$$

where γ_i^l denotes the scaling factor in group i of layer l, β_i^l denotes the bias factor, and I_i^l represents the pruning sensitivity score associated with this parameter group. Since the influence of the scaling and the bias factors on pruning sensitivity may differ across various networks and tasks, a tunable parameter μ is introduced to control the relative weighting of these two factors for more flexible and accurate sensitivity estimation. Specifically, when $\mu < 0.5$, the bias factor β is considered more influential; when $\mu > 0.5$, the scaling factor γ is given greater weight. When $\mu = 0.5$, both factors contribute equally to the pruning sensitivity score. By default, μ is set to 0.5, but it can be adjusted according to the specific requirements of the pruning task.

After computing the pruning sensitivity scores for all BN parameter groups, a global pruning rate R is applied to determine the pruning ratio for each layer. This facilitates cross-layer comparison of sensitivity scores and ensures that at least one filter is retained in every layer. The pruning sensitivity score is subjected to min-max normalization as follows:

$$U_i^l = \frac{I_i^l - I_{\min}^l}{I_{\max}^l - I_{\min}^l}, \tag{13}$$

where $I_{\min}^l$ and $I_{\max}^l$ denote the minimum and maximum pruning sensitivity scores respectively, among all BN parameter groups in the layer l. The normalized pruning sensitivity score of the i-th BN parameter group in layer l is denoted as U_i^l, which is computed using min-max normalization. This normalized score U_i^l allows for consistent comparison of pruning sensitivities across different layers. Since the sensitivity scores have been normalized to the range $[0, 1]$, it follows that when $I_i^l = I_{\min}^l$, we have $U_i^l = 0$, and when $I_i^l = I_{\max}^l$, $U_i^l = 1$. This normalization ensures that pruning decisions do not result in the removal of all filters within any individual layer. Assuming that the l-th convolution layer contains N_l filters and R denote the global pruning rate, the total number of filters to be pruned from the network can be calculated as:

$$P = R\sum_{i=1}^{L} N_i. \tag{14}$$

After obtaining the global normalized pruning sensitivity scores U_i^l, the global pruning rate R is employed to determine the pruning rate for each convolution layer. The allocation procedure proceeds as follows: all global normalized pruning sensitivity scores U_i^l are first sorted in ascending order, as shown in Eq. (15).

$$U_{i_1}^l \le U_{i_2}^l \le \cdots \le U_{i_p}^l \le \cdots \le U_{i_N}^l, \tag{15}$$

where $U_{i_p}^l$ denotes the pruning sensitivity score at the p-th position in the sorted sequence, where a higher score indicates greater sensitivity to pruning. Next, a pruning sensitivity flag is assigned: scores positioned before p are flagged with 1, indicating insensitivity to pruning, while scores after p are flagged with 0, indicating sensitivity to pruning, as illustrated in Eq. (16).

$$F_i^l = \begin{cases} 0, & U_i^l > U_{ip}^l \\ 1, & U_i^l \le U_{ip}^l \end{cases}, \tag{16}$$

$$p_l = \frac{\sum_{i=1}^{N_l} F_i^l}{N_l}. \tag{17}$$

Finally, the pruning rate for each convolutional layer is computed, as shown in Eq. (17), where p_l represents the pruning rate for the l-th layer.

4.5. Fine-tune with knowledge distillation

After determining the pruning rate p_l for each layer, the number of output channels to be pruned in the l-th convolution layer is computed as:

$$P_l = N_l \cdot p_l, \tag{18}$$

where N_l denotes the total number of output channels in the l-th layer, p_l is the pruning rate for that layer, and P_l represents the number of channels to be pruned. To identify which specific channels should be removed, the channel importance scores within each layer are sorted in ascending order:

$$SV_{i_1}^l \le SV_{i_2}^l \le \cdots \le SV_{i_{P_l}}^l \le \cdots \le SV_{i_{N_l}}^l, \tag{19}$$

where $SV_{i_p}^l$ denotes the channel importance score at the p-th position in the sorted list, where

$$p = P_l. \tag{20}$$

Channels with scores below $SV_{t_p}^l$ will be pruned. A pruning mask M_n^l is then defined to indicate whether each channel should be retained:

$$M_n^l = \begin{cases} 1, & \text{if } s_n^l \geq s_p^l \\ 0, & \text{if } s_n^l < s_p^l \end{cases}, \tag{21}$$

where M_n^l is a binary mask indicating whether the n-th channel in the l-th layer should be preserved. If $M_n^l = 1$, the channel is retained; if $M_n^l = 0$, the channel is pruned.

During the fine-tuning stage following pruning, knowledge distillation is incorporated to further enhance model performance. Knowledge distillation is a widely used model compression technique in deep learning, which aims to transfer knowledge from a large and complex model (the teacher model) to a smaller and more efficient model (the student model). The core principle of knowledge distillation lies in utilizing the output probability distribution of the teacher model as soft labels during the training process of the student model, rather than relying solely on hard labels derived from the original dataset. This strategy allows the student model to capture the fine grained representations and decision making behaviors of the teacher model, rather than learning only the ground truth targets, thereby significantly improving the generalization and performance of the student model.

The loss function for knowledge distillation comprises two components: the hard label loss and the soft label loss. The hard label loss is the standard cross-entropy loss, which measures the discrepancy between the student model's predictions and the ground truth labels. It is defined as follows:

$$\mathcal{L}_{\text{hard}}(y, q) = -\sum_i y_i \log q_i, \tag{22}$$

where y_i is the ground truth label for the i-th sample, and q_i denotes the predicted probability from the student model for that sample. The soft label loss is based on the output distributions of the teacher model (p_i) and the student model (q_i), using the Kullback-Leibler (KL) divergence to quantify the difference between them. It is defined as:

$$\mathcal{L}_{\text{soft}}(p, q) = T^2 \cdot \text{KL}\left(\text{softmax}\left(\frac{p}{T}\right) \middle\| \text{softmax}\left(\frac{q}{T}\right)\right) = T^2 \sum_i \left(\frac{p_i}{T} \log \frac{p_i/T}{q_i/T}\right), \tag{23}$$

where the KL denotes the KL divergence, which represents the loss incurred when approximating the soft labels of the teacher model using the student model's output distribution. A larger KL value indicates a worse approximation. A temperature parameter T is introduced to smooth the softmax outputs: when $T > 1$, the output distribution becomes more uniform; when $T = 1$, it reduces to the standard softmax. Since the gradient of the softmax function is affected by a factor of $\frac{1}{T}$, the loss is scaled by T^2 to ensure consistent gradient magnitudes during backpropagation. The final loss function is a weighted combination of the two components:

$$\mathcal{L} = \lambda \mathcal{L}_{\text{soft}}(p, q) + (1 - \lambda)\mathcal{L}_{\text{hard}}(y, q), \tag{24}$$

where λ is a hyperparameter that balances the contributions of the soft and hard label losses. Typically, λ is set to favor the soft label loss, as it carries richer information from the teacher model.

Although knowledge distillation is a widely used technique for model compression, it requires selecting appropriate teacher and student models. One limitation is the absence of clear guidance on how to choose an effective lightweight student model. To address this issue, this study adopts a pruned version of the original network as the student model, while retaining the original network as the teacher model. This approach effectively bridges the gap in student model selection and combines the benefits of both pruning and distillation, resulting in improved model performance.

As shown in Algorithm 1, the CLBM pruning algorithm follows a pipeline to compress DCNNs. It begins by training a baseline model and then identifies significant convolution kernels based on L_1 norm statistics. A cross-layer bidirectional metric is used to evaluate channel importance from both generation and distribution perspectives, shown in lines 3–8. Pruning rates are adaptively determined by analyzing BN parameters,shown in lines 9 and 10, and pruning is performed accordingly in line 12. Finally, knowledge distillation is applied to fine-tune the pruned model using the original model as the teacher, yielding a compact, high performance network suitable for deployment.

5. Experiment and analysis

In this section, we provide a detailed introduction to the dataset and network selected for the experiment, performance metrics, experimental settings, and experimental results.

5.1. Implementation details

Dataset: The datasets used in the experiment include CIFAR10, CIFAR100, and ImageNet (ISLVRC2012). CIFAR10 is a small, simple dataset containing 60,000 images of size 32×32, divided into 10 categories. Each category contains 6000 images, with 50,000 used for training and 10,000 for testing. CIFAR100 is a small, complex dataset comprising 60,000 images of 32×32 pixels, divided into 100 categories, with each category containing 600 images. The 100 categories are further divided into 20 superclasses, with each superclass containing 5 subcategories. ImageNet is a large, complex dataset comprising over 14 million images across thousands of categories. ISLVRC2012 is a subset of this dataset, containing approximately 1.2 million training images and 50,000 validation images, categorized into 1000 categories.

Algorithm 1 CLBM pruning framework.

Require: Pretrained model, global pruning ratio R
Ensure: Lightweight pruned model
Train a DCNN from scratch to obtain a pretrained model
for each convolution layer **do**
    Compute the L_1 norm of all convolution kernels
    Fit the norm distribution to get mean $E[X]$ and variance $Var(X)$;
    Select significant kernels using $T_l = E_l - c\sqrt{V_l}$ and Eq. (8)
    Compute synthesis perspective importance using $SG_n^l = \sum_{m=1}^{M_l} ||K_{n,m}^l||_1 \cdot D_{n,m}^l$
    Compute distribution perspective importance using $SR_n^l = \sum_{i=1}^{N_{l+1}} ||K_{i,n}^{l+1}||_1 \cdot D_{i,n}^{l+1}$
    Combine to get overall importance using Eq. (11)
    Extract BN parameters
    Compute pruning ratio using Eqs. (12) and (17)
end for
Prune channels using Eqs. (20) and (21),
Fine-tune the pruned model via knowledge distillation, using the original model as teacher
return Final lightweight high performance model

Network architecture: In terms of the selected networks, our experiment encompasses various types of networks. These include the single-branch network VGGNet, the multi-branch network ResNet, and the densely connected network DenseNet. In the CIFAR10 experiments, our method was validated using VGG16, ResNet56, and DenseNet40; in the CIFAR100 experiments, VGG16 was used for validation; for the large-scale dataset ImageNet (ISLVRC2012), ResNet50 was used for experimental validation. By conducting experiments with a wide range of datasets and networks, the effectiveness of our method is demonstrated.

Evaluation metrics: Same as existing mainstream methods, our method evaluates the compression and acceleration effects of DCNNs models using classification accuracy before and after pruning, the number of parameters, and the number of FLOPs. Under the premise that classification accuracy remains largely unchanged, the greater the reduction in the number of parameters and computational operations, the better the compression effect of the algorithm; under the premise that the compression or acceleration ratio is the same, the smaller the decrease in classification accuracy, the better the compression effect of the algorithm. For small datasets such as CIFAR10 and CIFAR100, classification accuracy evaluation selects Top-1; for large datasets such as ImageNet (ISLVRC2012), classification accuracy evaluation selects Top-1 and Top-5.

To compare the changes in the number of parameters and computation volume before and after pruning, we introduce the compression ratio (CR) and acceleration ratio (AR). These two metrics reflect the comparison of model performance before and after pruning, and their formulas are as follows:

$$CR = \frac{Parameters_{original}}{Parameters_{pruned}}, \tag{25}$$

$$AR = \frac{FLOPS_{original}}{FLOPS_{pruned}}, \tag{26}$$

where CR is the ratio of the number of parameters in the original model to the number of parameters in the pruned model. The larger this ratio, the more ideal the compression effect of the pruned model. AR is the ratio of the number of FLOPs in the original model to the number of FLOPs in the pruned model. The larger this value, the better the acceleration effect of the model.

Other settings: All experiments in our method were conducted using the PyTorch deep learning framework. For the CIFAR10 and CIFAR100 datasets, experiments were performed on a machine equipped with an NVIDIA GTX 3090 GPU (24 GB VRAM), an Intel(R) Core(TM) i9-10920X CPU, and the CUDA version 12.2. For the ImageNet (ISLVRC2012) dataset, experiments were conducted on a system with an NVIDIA RTX 4090 GPU (24 GB VRAM), an Intel(R) Xeon(R) Gold 6230R CPU, and the same CUDA version. The batch size was set to 64 for both CIFAR10 and CIFAR100 datasets, and 128 for the ImageNet (ISLVRC2012) dataset. Stochastic Gradient Descent (SGD) was used as the optimizer, with an initial learning rate of 0.1, momentum of 0.9, and weight decay of 10^{-4}. A learning rate decay strategy was employed during training. The parameter μ in Eq. (12) was set to 0.5 by default, and increased to 0.85 for pruning experiments on the ImageNet (ISLVRC2012) dataset. During the fine-tuning stage, knowledge distillation was applied with a temperature $T = 5.0$, and a balancing coefficient $\lambda = 0.7$. All convolution layers were followed by BN layers throughout the network.

5.2. Results on CIFAR10/100

On the CIFAR10 dataset, CLBM is evaluated on three network architectures: VGG16, ResNet56, and DenseNet40. The pruning results for the single-branch network VGG16 are summarized in Table 1. In this table, “Model” refers to the network architecture used in the experiment, while “Alg” indicates the pruning algorithm applied, with CLBM representing the method proposed in this study. The performance of CLBM is compared with several state-of-the-art (SOTA) approaches, including classical methods such as FPGM and HRank, as well as recent methods proposed by Zheng et al. and Guo et al. To ensure fairness and reproducibility, all

Table 1
CLBM pruning experiment results for the VGG16 network on the CIFAR10 dataset.

Model	Alg	Acc (%)	Acc↓ (%)	Parameters/CR(x)	FLOPs/AR(x)
VGG16	SSS [27] (2018)	93.96 → 93.02	0.94	3.93 M/3.73	183.13 M/1.71
	GAL-0.05 [32] (2019)	93.96 → 92.03	1.93	3.36 M/4.39	189.49 M/1.66
	FPGM [18] (2019)	93.58 → 93.23	0.35	–	201.11 M/1.56
	VCNNP [33] (2019)	93.25 → 93.18	0.07	3.92 M/3.75	190.01 M/1.65
	HRank [22] (2020)	93.96 → 92.34	1.62	2.64 M/5.57	108.61 M/2.88
	Li et al. [34] (2023)	93.96 → 92.59	1.37	1.38 M/10.67	125.25 M/2.50
	Zheng et al. [35] (2024)	93.83 → 93.48	0.35	–	114.52 M/2.74
	Guo et al. [36] (2024)	93.64 → 92.52	1.12	1.36 M/10.82	62.31 M/5.04
	CLBM	**93.70 → 93.53**	**0.17**	**1.10 M/13.38**	**59.52 M/5.27**

Table 2
CLBM pruning experiment results for the ResNet56 network on the CIFAR10 dataset.

Model	Alg	Acc (%)	Acc↓ (%)	Parameters/CR(x)	FLOPs/AR(x)
ResNet56	GAL-0.6 [32] (2019)	93.26 → 92.98	0.28	0.75 M/1.13	78.30 M/1.62
	FPGM [18] (2019)	93.59 → 93.26	0.33	–	59.40 M/2.13
	HRank [22] (2020)	93.26 → 93.17	0.09	0.49 M/1.73	62.72 M/2.02
	Feng et al. [37] (2022)	93.30 → 93.05	0.25	0.67 M/1.27	63.78 M/1.98
	Li et al. [34] (2023)	93.26 → 92.88	0.38	0.57 M/1.49	80.77 M/1.57
	DDFP [38] (2023)	93.59 → 93.34	0.25	0.41 M/2.07	59.40 M/2.13
	Zheng et al. [35] (2024)	93.64 → 93.04	0.60	–	48.72 M/2.60
	Guo et al. [36] (2024)	93.62 → 93.54	0.08	0.38 M/2.24	49.50 M/2.56
	CLBM	**94.22 → 94.36**	**−0.14**	**0.32 M/2.83**	**44.41 M/2.85**

baseline results are reported as originally published in their respective papers. As shown in Table 1, at an overall pruning rate of 0.67, CLBM achieves a substantial reduction in both computational cost and parameter count while maintaining high classification accuracy. After pruning, VGG16 with CLBM contains only 1.10 M parameters and 59.52 M FLOPs, corresponding to a compression ratio of 13.38×and an acceleration ratio of 5.27×, with an accuracy drop of merely 0.17%. Compared with the HRank, CLBM achieves a 1.45% smaller drop in accuracy, uses 1.54 M fewer parameters, and reduces FLOPs by 49.09 M. Compared to the latest method by Guo et al., CLBM improves accuracy by 0.95%, reduces parameters by 0.26 M, and saves 2.79 M FLOPs. Overall, CLBM demonstrates superior performance in terms of compression ratio, acceleration ratio, and accuracy retention. These results indicate that CLBM is highly effective for compressing and accelerating the single-branch VGG16 network on the relatively simple CIFAR10 dataset.

The pruning results for the multi-branch network ResNet56 are presented in Table 2. The proposed CLBM method is compared with several SOTA approaches, including classical methods such as FPGM and HRank, as well as more recent methods like DDFP, Zheng et al., and Guo et al. As shown in Table 2, CLBM achieves substantial reductions in both computational complexity and parameter count, while maintaining or even slightly improving classification accuracy compared to the original model. After pruning with CLBM, the ResNet56 network contains 0.32 M parameters and 44.41 M FLOPs, corresponding to a compression ratio of 2.83×, an acceleration ratio of 2.85×, and an accuracy improvement of 0.14%. Compared with the FPGM algorithm, CLBM achieves a 0.47% smaller accuracy drop and reduces FLOPs by 14.99 M. Under similar accuracy conditions, CLBM outperforms FPGM in terms of both accuracy retention and computational efficiency. Compared with the recent method by Guo et al., CLBM yields a 0.22% smaller accuracy drop, with 0.06 M fewer parameters and 5.09 M fewer FLOPs. Across all listed methods, CLBM consistently demonstrates notable advantages in compression ratio, acceleration ratio, and accuracy preservation. These results indicate that CLBM is highly effective in compressing and accelerating the multi-branch networks like ResNet56 on the relatively simple CIFAR10 dataset.

The pruning results for the multi-branch densely connected network DenseNet40 are shown in Table 3. The proposed CLBM method is compared with several SOTA techniques, including classical methods such as GAL and HRank, as well as recent approaches like FSIM-E and CORING. As shown in Table 3, CLBM achieves substantial reductions in both computational complexity and parameter count, while preserving the model's accuracy with minimal degradation. After pruning, the DenseNet40 network retains only 0.44 M parameters and 120.64 M FLOPs, resulting in a compression ratio of 2.41×and an acceleration ratio of 2.40×, with a 0.13% increase in accuracy compared with the original model. In comparison to HRank, CLBM yields a 0.70% smaller accuracy drop, with 0.22 M fewer parameters and 46.77 M fewer FLOPs. Against FSIM-E, CLBM achieves a 0.43% lower accuracy drop, 0.15 M fewer parameters, and 44.72 M fewer FLOPs. Compared with the recent CORING algorithm, CLBM demonstrates a 0.64% lower accuracy drop, with 0.01 M fewer parameters and 14.22 M fewer FLOPs. Under comparable compression ratios, CLBM consistently outperforms these baselines in terms of accuracy retention and computational efficiency. These results demonstrate the effectiveness of CLBM in compressing and accelerating the DenseNet40 network on the relatively simple CIFAR10 dataset.

On the CIFAR100 dataset, CLBM is evaluated through pruning experiments on the VGG16 network. The corresponding results are presented in Table 4. The proposed method is compared against several advanced pruning approaches, including classical algorithms such as VCNNP and HRank, as well as more recent methods like that of Guo et al. As shown in Table 4, CLBM achieves substantial reductions in both computational cost and parameter count while maintaining or even improving classification accuracy. After pruning, the VGG16 network retains 3.91 M parameters and 144.83 M FLOPs, resulting in a compression ratio of 3.76×, an acceleration ratio of 2.17×, and an accuracy gain of 0.51%. Compared with the HRank, CLBM achieves a 1.61% higher accuracy, with 0.20 M

Table 3
CLBM pruning experiment results for the DenseNet40 network on the CIFAR10 dataset.

Model	Alg	Acc (%)	Acc↓ (%)	Parameters/CR(x)	FLOPs/AR(x)
DenseNet40	GAL-0.01 [32] (2019)	94.81 → 94.29	0.52	0.67 M/1.58	182.92 M/1.58
	HRank [22] (2020)	94.81 → 94.24	0.57	0.66 M/1.60	167.41 M/1.73
	FSIM-E [39] (2023)	94.72 → 94.42	0.30	0.59 M/1.80	165.36 M/1.75
	CORING [40] (2024)	94.81 → 94.30	0.51	0.45 M/2.36	134.86 M/2.15
	CLBM	**94.28 → 94.41**	**−0.13**	**0.44 M/2.41**	**120.64 M/2.40**

Table 4
CLBM pruning experiment results for the VGG16 network on the CIFAR100 dataset.

Model	Alg	Acc (%)	Acc↓ (%)	Parameters/CR(x)	FLOPs/AR(x)
VGG16	VCNNP [33] (2019)	73.26 → 73.33	−0.07	9.14 M/1.61	256.00 M/1.23
	CPGMI [41] (2020)	73.26 → 73.53	−0.27	4.99 M//2.94	198.20 M/1.58
	HRank [22] (2020)	73.55 → 72.45	1.10	4.11 M/3.58	162.83 M/1.93
	Guo et al. [36] (2023)	72.44 → 72.78	−0.34	–	172.56 M/1.82
	CLBM	**73.47 → 73.98**	**−0.51**	**3.91 M/3.76**	**144.83 M/2.17**

Table 5
CLBM pruning experiment results for the ResNet50 network on the ImageNet (ISLVRC2012) dataset.

Model	Alg	Top-1%	Top-1 ↓ (%)	Top-5%	Top-5 ↓ (%)	FLOPs/AR(x)	Parameters/CR(x)
ResNet50	SFP [15] (2018)	76.15 → 62.14	14.01	92.87 → 84.60	8.27	2.38B/1.72	–
	SSS-32 [27] (2018)	76.12 → 74.18	1.94	92.86 → 91.91	0.95	2.82B/1.45	18.60 M/1.37
	SSS-26 [27] (2018)	76.12 → 71.82	4.30	92.86 → 90.79	2.07	2.33B/1.76	15.60 M/1.63
	GAL-1 [32] (2019)	76.15 → 69.88	6.27	92.87 → 89.75	3.12	1.58B/2.59	14.67 M/1.74
	SSR [42] (2019)	–	3.97	–	2.01	1.70B/2.41	12.20 M/2.09
	RBP [43] (2019)	76.10 → 71.50	4.60	92.90 → 90.20	2.70	1.78B/2.30	–
	FPGM [18] (2019)	76.15 → 74.83	1.32	92.87 → 92.32	0.55	1.90B/2.15	–
	HRank [22] (2020)	76.15 → 71.98	4.17	92.87 → 91.01	1.86	1.55B/2.64	13.77 M/1.86
	EACP [44] (2023)	75.82 → 73.42	2.40	92.89 → 91.21	1.68	1.73B/2.36	10.91 M/2.33
	AFPruner [45] (2024)	76.11 → 74.23	1.88	92.96 → 91.89	1.07	2.06B/1.98	12.00 M/2.13
	CLBM	**75.74 → 74.25**	**1.49**	**92.68 → 91.95**	**0.73**	**1.68B/2.43**	**9.63 M/2.65**

fewer parameters and 18.00 M fewer FLOPs, demonstrating superior performance across all evaluation metrics. Compared with the method by Guo et al., CLBM yields a 0.17% higher accuracy and reduces FLOPs by 27.73 M. Notably, while improving accuracy, CLBM also delivers a significantly higher acceleration ratio than Guo et al.'s approach.

These results indicate that CLBM delivers effective compression and acceleration on the challenging CIFAR100 dataset, outperforming both classical and recent SOTA pruning algorithms.

5.3. Results on the ImageNet (ISLVRC2012)

On the ImageNet (ISLVRC2012) dataset, CLBM is evaluated through pruning experiments on the ResNet50 network. The corresponding results are presented in Table 5. Several advanced pruning algorithms were selected for comparison, including classical methods such as RBP and HRank, and recent approaches such as EACP and AFPruner. As shown in Table 5, CLBM maintains a clear overall advantage in terms of Top-1 and Top-5 accuracy retention, compression ratio, and acceleration ratio. After pruning, the ResNet50 retains 9.63 M parameters and 1.68B FLOPs, achieving a compression ratio of 2.65×and an acceleration ratio of 2.43×. The Top-1 accuracy drops by only 1.49%, and the Top-5 accuracy by 0.73%. Compared with the HRank, CLBM achieves a 2.68% smaller drop in Top-1 accuracy, a 1.13% smaller drop in Top-5 accuracy, and reduces the parameter count by 4.14 M. These results demonstrate that CLBM significantly outperforms HRank in both accuracy retention and compression. Compared with the EACP, CLBM yields a 0.91% lower Top-1 accuracy drop, a 0.95% lower Top-5 accuracy drop, 1.28 M fewer parameters, and 0.05B fewer FLOPs. Across all evaluation metrics, CLBM outperforms the EACP algorithm. When compared with the recent AFPruner method, CLBM achieves a 0.39% smaller drop in Top-1 accuracy, a 0.34% smaller drop in Top-5 accuracy, reduces the parameter count by 2.37 M, and decreases FLOPs by 0.38B. These results confirm that CLBM consistently outperforms AFPruner in terms of accuracy, compression, and computational efficiency.

Overall, the experimental results indicate that CLBM achieves excellent pruning performance on the large scale and complex ImageNet (ISLVRC2012) dataset, offering superior compression and acceleration while maintaining high classification accuracy.

6. Ablation analysis

6.1. Impact of individual modules on pruning performance

To evaluate the effectiveness of the three modules proposed in CLBM, comprehensive ablation experiments were conducted, comprising two main components. The first part examines the contributions of the "significant convolution kernels in convolution

Table 6
Results of ablation experiments on VGG16 using CLBM on CIFAR10(1).

LSCK	AP	Baseline (%)	Pruned (%)	Acc↓ (%)	Parameters/CR	FLOPs/AR
×	×	93.70	91.63	2.07	1.59 M/9.26	34.42 M/9.12
✓	×		91.90	1.80	1.59 M/9.26	34.42 M/9.12
×	✓		93.07	0.63	1.10 M/13.38	59.52 M/5.27
✓	✓		93.45	0.25	1.10 M/13.38	59.52 M/5.27

Table 7
Results of ablation experiments on VGG16 using CLBM on CIFAR10(2).

Synthesis	Distribution	Baseline (%)	Pruned (%)	Acc↓ (%)	Parameters/CR	FLOPs/AR
×	×	93.70	92.98	0.72	1.10 M/13.38	59.52 M/5.27
✓	×		93.45	0.25		
×	✓		93.22	0.48		
✓	✓		93.53	0.17		

layers" and the "adaptive pruning rate" modules. For the filter importance metric based on significant convolution kernels, the control group uses all convolution kernel parameters. For the adaptive pruning rate module, the control group applies a uniform pruning rate across all convolution layers based on BN layer parameters. All experiments employ knowledge distillation during fine-tuning. The VGG16 model on the CIFAR10 dataset is selected as a representative case to assess the influence of the two modules on the performance of the pruned model. The corresponding results are summarized in Table 6.

As shown in Table 6, when the pruning rate allocation strategy proposed by CLBM is not employed, the model accuracy drops significantly due to the neglect of layer wise pruning sensitivity. Under this setting, using the significant convolution kernel evaluation yields a pruned model with 0.27% higher accuracy than the model pruned using all parameters, confirming the effectiveness of the proposed kernel selection strategy. Moreover, when the pruning rate allocation mechanism is applied—regardless of whether significant convolution kernels are selected—the resulting model consistently outperforms the one obtained without pruning rate allocation. This demonstrates the importance of adapting the pruning rate based on BN layer statistics. The best performance is achieved when both the significant convolution kernel selection and the pruning rate allocation modules are used in combination. This result validates the complementary nature and necessity of the two components proposed in CLBM.

The second part of the ablation study evaluates the effectiveness and necessity of incorporating cross-layer bidirectional channel information as a joint importance metric. Four experimental groups are constructed for comparison: (1) random pruning without any importance metric, (2) pruning using only the synthesis perspective importance metric, (3) pruning using only the distribution perspective importance metric, and (4) pruning using the proposed joint metric that combines both synthesis perspective and distribution perspective importance. All groups adopt the fully parameterized BN layer pruning rate allocation strategy proposed in this paper. The experimental results are summarized in Table 7.

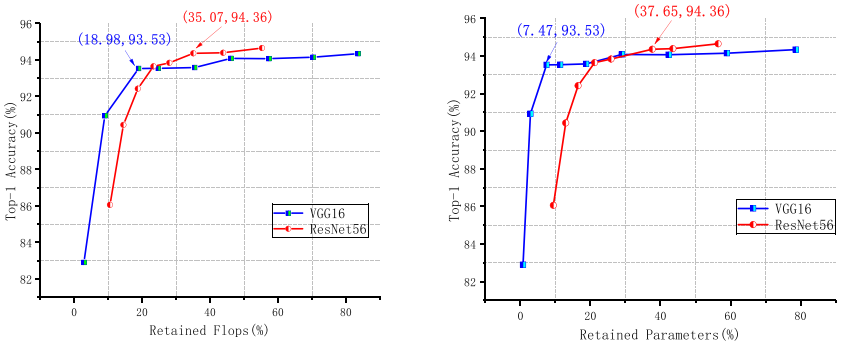
As shown in Table 7, when the compression and acceleration ratios are held constant, the accuracy drops significantly if pruning is performed without any importance metric. In this case, the model's accuracy declines from 93.7% to 92.98%, representing a reduction of 0.72%. When only the synthesis perspective importance metric is applied, the accuracy drop is reduced to 0.25%, which is 0.47% lower than that observed with random pruning. This suggests that the use of importance metrics allows for more effective identification of redundant filters and leads to improved pruning performance. When only the distribution perspective importance metric is used, the accuracy decrease is 0.48%, lying between the results of random pruning and synthesis perspective pruning. This outcome indicates that, in a single layer evaluation context on VGG16 with CIFAR10, the output quality of the current layer is more critical than the input information distribution of the subsequent layer. When both the synthesis perspective and distribution perspective importance metrics are jointly considered, the accuracy drop is minimized to just 0.17%, outperforming the single metric approaches. These results highlight the effectiveness and necessity of cross-layer bidirectional channel importance evaluation.

In conclusion, all three modules proposed in this work positively contribute to model performance, validating their effectiveness and necessity in the pruning framework.

6.2. Influence of compression ratios

In this subsection, we evaluate how different compression ratios affect accuracy on VGG16 and ResNet56 in CIFAR10. As shown in Fig. 10, both FLOPs and parameter pruning exhibit similar trade-off behavior: accuracy decreases under aggressive compression but stabilizes once the retained ratio reaches a moderate level.

For VGG16, accuracy drops sharply when the retained FLOPs fall below 10%, but quickly recovers and remains stable beyond this point. The model still achieves 93.53% accuracy with only 18.98% FLOPs and 7.47% parameters retained, which we identify as the best trade-off point. In contrast, ResNet56 shows a smoother degradation curve, but its turning point occurs at a higher retention level, with the optimal balance achieved at approximately 35.07% FLOPs and 37.65% parameters retained, where the accuracy remains at 94.36%. These results indicate that both networks contain considerable redundancy, though their sensitivity to pruning differs across architectures. Accordingly, the annotated best trade-off points in Fig. 10 are adopted as the default pruning settings, as they provide substantial compression while maintaining stable accuracy.



(a) Impact of FLOPS pruning rates.

(b) Impact of parameter pruning rates.

Fig. 10. Impact of different compression ratios in VGG16 and ResNet56.

6.3. Influence of μ

In this subsection, we further examine the influence of the hyperparameter μ on the layer-wise pruning behavior. As defined earlier, μ controls the weighting between the scaling factor γ and the bias factor β, where $\mu < 0.5$ emphasizes β and $\mu > 0.5$ prioritizes γ, while $\mu = 0.5$ assigns equal importance to both. To verify its effect in practice, we conduct experiments on VGG16 and ResNet56 using CIFAR10, with μ ranging from 0.1 to 0.9. The results are visualized in Fig. 11, where the horizontal axis represents the index of convolution layers and the vertical axis denotes the retained ratio.

For VGG16, the curve corresponding to $\mu = 0.1$ shows noticeably larger fluctuations and lower retention in several layers, leading to suboptimal overall accuracy. When $\mu \geq 0.3$, the layer retained rates become more stable, and the accuracy converges around 93.5%. This confirms that extremely small μ values, which overemphasize the bias factor, may lead to excessively aggressive pruning in early layers. In contrast, the results for ResNet56 demonstrate much smaller performance variations across different values of μ, and all configurations achieve comparable accuracy around 94.3%. This suggests that deeper residual architectures are more robust to the choice of μ, as the pruning impact is distributed across many layers.

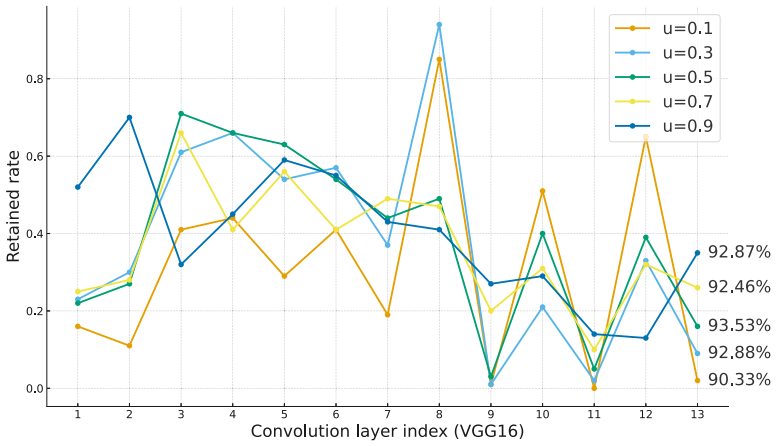
Based on these observations, we adopt $\mu = 0.5$ for both architectures in CIFAR10, which assigns equal importance to γ and β and yields stable performance. This default setting also aligns with the theoretical definition of μ, ensuring a balanced pruning sensitivity measure.

6.4. Cross-layer visualization analysis

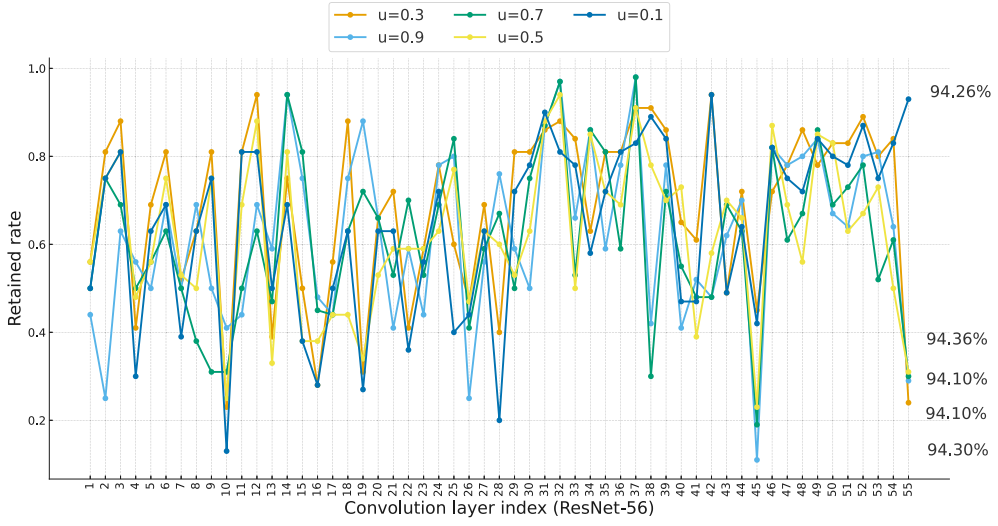
To visualize the information propagation across layers, we conduct a cross-layer channel analysis of ResNet. Here, F denotes the output feature maps of the preceding layer, serving as the input activations of the current layer. The mean standard deviation of F, denoted as $\sigma(F)$, represents the spatial fluctuation of activations where larger fluctuations indicate stronger and more diverse responses, corresponding to higher signal energy and spatial activity. The weight norm $\|W\|$ refers to the kernels of the current layer and measures their transformation or amplification capability. A larger weight norm implies the stronger modulation or amplification to the input signal, whereas a smaller norm indicates weaker responses and potential redundancy in the channel. To jointly capture the effects of input activation and structural transformation, we define the cross-layer transmission strength as $S = \sigma(F) \cdot \|W\|$. This quantity characterizes the information propagation between consecutive layers, integrating both synthesis and distribution perspectives. A large S indicates that a channel efficiently information propagation, in contrast, a small S reflects weak propagation, suggesting that it can be regarded as a potential pruning candidate.

As shown in Fig. 12, we visualized three cross-layers from ResNet56. The horizontal axis denotes the activation strength from the synthesis perspective, while the vertical axis corresponds to the weight norm from the distribution perspective. The color of each point encodes the transmission value $S = \sigma(F) \cdot \|W\|$, where a brighter color indicates the effective channel. A consistent phenomenon can be observed across the three subgraphs: some channels exhibit high activation from the synthesis perspective but nearly zero weight norm from the distribution. In these cases, rich input information fails to propagate through the distribution, leading to a discontinuity in information flow between consecutive layers. This observation provides visual evidence for the motivation of our proposed method which considers both the synthesis and distribution perspectives to maintain consistent information transmission across layers.

Fig. 13 visualizes the information transmission in heatmap form. Each subgraph dedicates one layer example, the first column shows the weight distribution on the distribution, the second column presents the activation strength histogram on the synthesis, and the third column illustrates the integrated $S = \sigma(F) \cdot \|W\|$. It can be observed that in the S-value heatmaps, several regions exhibit significant attenuation compared with the weight maps. For instance, in layers 11 and 12, the values in rows 10 to 20 and 30 to 50 nearly decay to zero. This indicates that part of the information disappears during propagation. The visualization further demonstrates that evaluating channels from only one side is insufficient, and both the synthesis and distribution perspectives must be considered to ensure consistent information transmission.

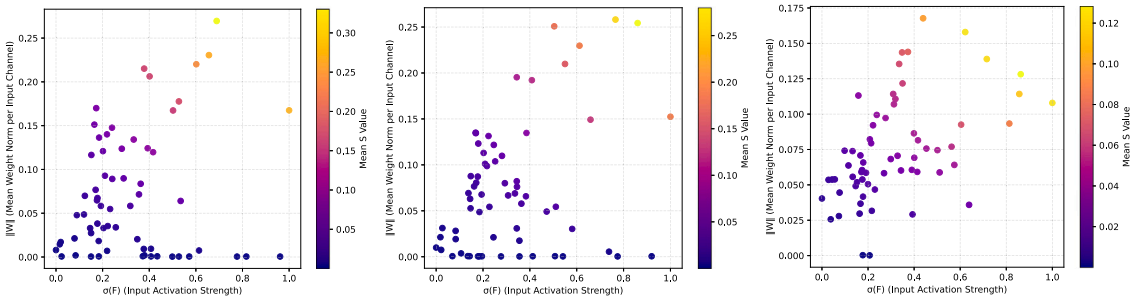


(a) Impact of μ in VGG16.



(b) Impact of μ in ResNet56.

Fig. 11. Impact of different compression ratios in VGG16 and ResNet56.



(a) Layer 8–9 (b) Layer 11–12 (c) Layer 20–21

Fig. 12. Cross-layer visualization of ResNet56 in scatter plot.

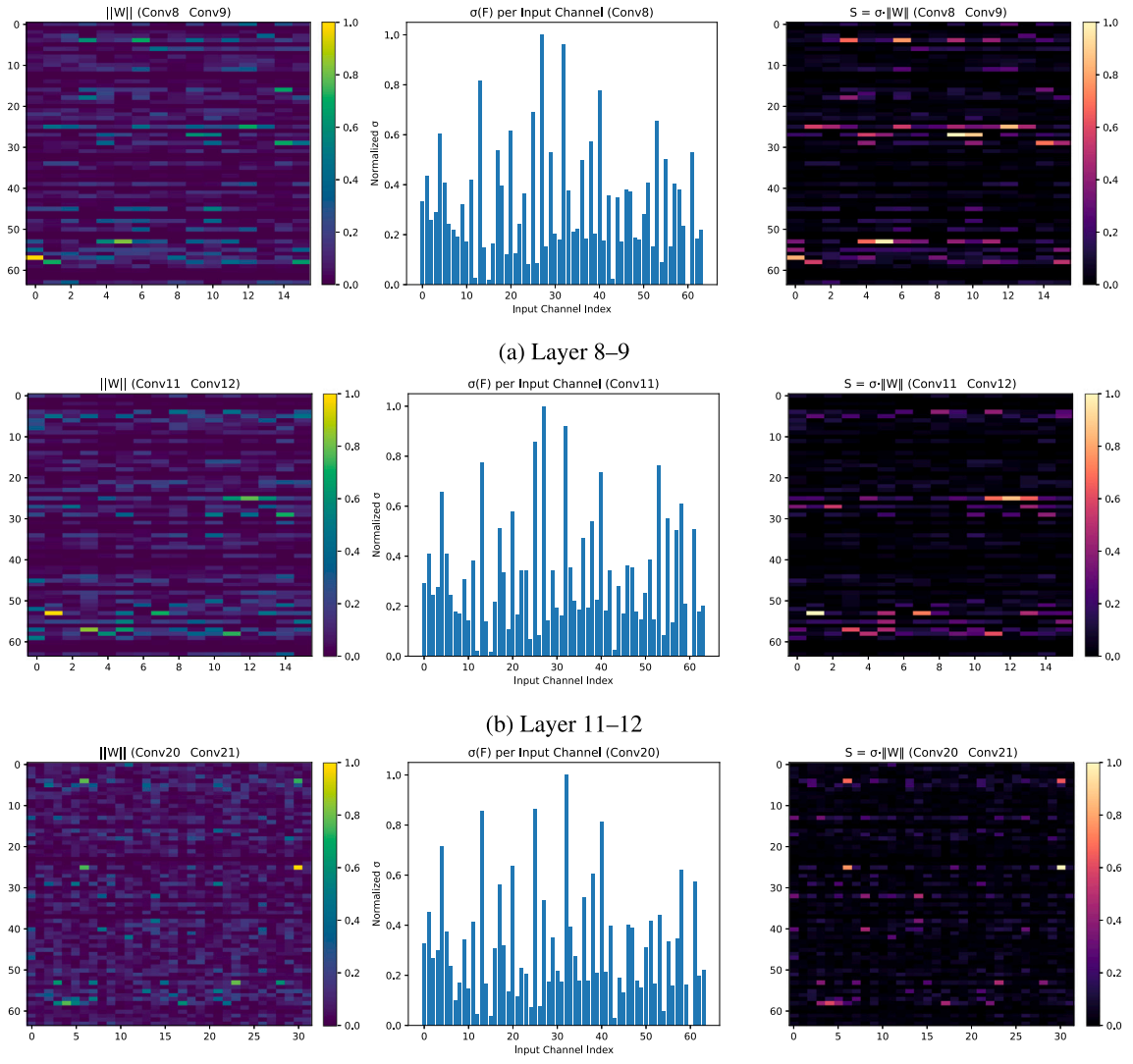


Fig. 13. Visualization of information flow discontinuity.

7. Conclusion

CLBM is an efficient pruning method for DCNNs that significantly reduces model parameters and FLOPs. By leveraging significant convolution kernel parameters for importance evaluation, CLBM effectively suppresses the interference introduced by low contribution kernels. Through cross-layer analysis, it provides a more comprehensive assessment of channel importance. For pruning rate determination, CLBM introduces an adaptive strategy based on the full parameter allocation of BN layers, thereby reducing the subjectivity associated with manually setting layer wise pruning rates. In the fine-tuning stage, knowledge distillation is seamlessly integrated to enhance model performance post pruning. Extensive experimental results validate the effectiveness of CLBM across various scenarios.

Despite its promising performance, there remains scope for further improvement of CLBM. Future research could focus on refining the selection criteria for significant convolution kernels, as a more precise identification of informative kernels can mitigate the noise introduced by less important ones during importance estimation. Additionally, while CLBM demonstrates strong compression and acceleration capabilities on diverse DCNNs architectures, future extensions could explore its applicability to modern network designs such as transformer based architectures, enabling more efficient operation in complex tasks. This study has validated the method across multiple general purpose datasets, achieving promising results. To promote practical deployment, future work may involve adaptation to specific application domains (e.g., autonomous driving), including the development of domain specific datasets and targeted validation, thereby facilitating real world adoption and technological transfer of the proposed approach.

CRediT authorship contribution statement

Suyun Lian: Writing – original draft. **Jiajian Cai:** Writing – original draft. **Yang Zhao:** Writing – review & editing. **Stefan Poslad:** Writing – review & editing. **Jihong Pei:** Writing – review & editing.

Ethical and informed consent for data used

No ethical approval or informed consent was necessary for this study, as the data were already publicly available and did not involve human or animal subjects.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported in part by the National Natural Science Foundation of China (Grant No. 62201355), Guangdong Basic and Applied Basic Research Foundation (2024A1515010977, 2025A1515060013), Scientific Foundation for Youth Scholars of Shenzhen University, Guangdong Provincial Key Laboratory (2023B1212060076, 2017B030314073).

Data availability

Data will be made available on request.

References

[1] H. Cheng, M. Zhang, J.Q. Shi, A survey on deep neural network pruning: taxonomy, comparison, analysis, and recommendations, IEEE Trans. Pattern Anal. Mach. Intell. 46 (12) (2024) 10558–10578.
[2] K. Sharifani, M. Amini, Machine learning and deep learning: a review of methods and applications, World Inf. Technol. Eng. J. 10 (7) (2023) 3897–3904.
[3] Y. He, L. Xiao, Structured pruning for deep convolutional neural networks: a survey, IEEE Trans. Pattern Anal. Mach. Intell. 46 (5) (2023) 2900–2919.
[4] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz, D. Terzopoulos, Image segmentation using deep learning: a survey, IEEE Trans. Pattern Anal. Mach. Intell. 44 (7) (2021) 3523–3542.
[5] K. Gkrispanis, N. Gkalelis, V. Mezaris, Filter-pruning of lightweight face detectors using a geometric median criterion, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) Workshops, 2024, pp. 280–289.
[6] O. Ronneberger, P. Fischer, T. Brox, U-net: convolutional networks for biomedical image segmentation, in: Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18, Springer, 2015, pp. 234–241.
[7] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, et al., Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 2704–2713.
[8] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, arXiv preprint arXiv:1503.02531, 2015.
[9] X. Ruan, Y. Liu, C. Yuan, B. Li, W. Hu, Y. Li, et al., EDP: an efficient decomposition and pruning scheme for convolutional neural network compression, IEEE Trans. Neural Netw. Learn. Syst. 32 (10) (2020) 4499–4513.
[10] S. Han, J. Pool, J. Tran, W. Dally, Learning both weights and connections for efficient neural network, Adv. Neural Inf. Process. Syst. 28 (2015).
[11] S. Han, H. Mao, W.J. Dally, Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding, arXiv preprint arXiv:1510.00149, 2015.
[12] W. Wen, C. Wu, Y. Wang, Y. Chen, H. Li, Learning structured sparsity in deep neural networks, Adv. Neural Inf. Process. Syst. 29 (2016).
[13] A. Jayasimhan, P. Pabitha, Resprune: an energy-efficient restorative filter pruning method using stochastic optimization for accelerating CNN, Pattern recognit. 155 (2024) 110671.
[14] H. Li, A. Kadav, I. Durdanovic, H. Samet, H.P. Graf, Pruning filters for efficient convnets, in: International Conference on Learning Representations, 2017.
[15] Y. He, G. Kang, X. Dong, Y. Fu, Y. Yang, Soft filter pruning for accelerating deep convolutional neural networks, arXiv preprint arXiv:1808.06866, 2018.
[16] J. Zhu, J. Pei, Progressive kernel pruning CNN compression method with an adjustable input channel, Appl. Intell. 52 (2022) 10519–10540.
[17] J. Back, N. Ahn, J. Kim, Magnitude attention-based dynamic pruning, Expert Syst. Appl. 276 (2025) 126957.
[18] Y. He, P. Liu, Z. Wang, Z. Hu, Y. Yang, Filter pruning via geometric median for deep convolutional neural networks acceleration, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 4340–4349.
[19] J. Pei, Z. Huang, J. Zhu, Pruning rate-controlled filter order–information structure similarity graph clustering for DCNN structure optimization methods, Multim. Tools Appl. 83 (32) (2024) 78407–78431.
[20] J. Chang, L. Tao, B. Lyu, X. Zhu, S. Liu, Q. Zou, et al., Bap: bilateral asymptotic pruning for optimizing CNNS on image tasks, Inf. Sci. 678 (2024) 120898.
[21] G. Li, R. Li, T. Li, C. Shen, X. Zou, J. Wang, et al., Sfp: similarity-based filter pruning for deep neural networks, Inf. Sci. 689 (2025) 121418.
[22] M. Lin, R. Ji, Y. Wang, Y. Zhang, B. Zhang, Y. Tian, et al., Hrank: filter pruning using high-rank feature map, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 1529–1538.
[23] S. Lian, Y. Zhao, J. Pei, Daar: dual attention cooperative adaptive pruning rate by data-driven for filter pruning, Appl. Intell. 55 (6) (2025) 1–17.
[24] J.-H. Luo, J. Wu, W. Lin, Thinet: a filter level pruning method for deep neural network compression, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 5058–5066.
[25] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, C. Zhang, Learning efficient convolutional networks through network slimming, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 2736–2744.
[26] T.-W. Chin, R. Ding, C. Zhang, D. Marculescu, Towards efficient model compression via learned global ranking, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 1518–1528.
[27] Z. Huang, N. Wang, Data-driven sparse structure selection for deep neural networks, in: Proceedings of the European Conference on Computer Vision (ECCV), 2018, pp. 304–320.
[28] X. Dong, P. Yan, M. Wang, B. Li, Y. Song, An optimization method for pruning rates of each layer in CNN based on the ga-smsm, Memet. Comput. 16 (1) (2024) 45–54.
[29] X. Wei, N. Zhang, W. Liu, H. Chen, Nas-based CNN channel pruning for remote sensing scene classification, IEEE Geosci. Remote Sens. Lett. 19 (2022) 1–5.
[30] B. Wang, V. Kindratenko, Rl-pruner: structured pruning using reinforcement learning for cnn compression and acceleration, arXiv preprint arXiv:2411.06463, 2024.

[31] Z. Liu, H. Mu, X. Zhang, Z. Guo, X. Yang, K.-T. Cheng, et al., Metapruning: meta learning for automatic neural network channel pruning, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 3296–3305.
[32] S. Lin, R. Ji, C. Yan, B. Zhang, L. Cao, Q. Ye, et al., Towards optimal structured CNN pruning via generative adversarial learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 2790–2799.
[33] C. Zhao, B. Ni, J. Zhang, Q. Zhao, W. Zhang, Q. Tian, Variational convolutional neural network pruning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 2780–2789.
[34] J. Li, H. Shao, S. Zhai, Y. Jiang, X. Deng, A graphical approach for filter pruning by exploring the similarity relation between feature maps, Pattern Recognit. Lett. 166 (2023) 69–75.
[35] Y. Zheng, P. Sun, Q. Ren, W. Xu, D. Zhu, A novel and efficient model pruning method for deep convolutional neural networks by evaluating the direct and indirect effects of filters, Neurocomputing 569 (2024) 127124.
[36] Y. Guo, W. Gao, G. Li, Interpretable task-inspired adaptive filter pruning for neural networks under multiple constraints, Int. J. Comput. Vis. 132 (6) (2024) 2060–2076.
[37] K.-Y. Feng, X. Fei, M. Gong, A.K. Qin, H. Li, Y. Wu, An automatically layer-wise searching strategy for channel pruning based on task-driven sparsity optimization, IEEE Trans. Circuits Syst. Video Technol. 32 (9) (2022) 5790–5802.
[38] G. Li, B. Liu, A. Chen, Ddfp: a data driven filter pruning method with pruning compensation, J. Vis. Commun. Image Represent. 94 (2023) 103833.
[39] Y. Liu, K. Fan, D. Wu, W. Zhou, Filter pruning by quantifying feature similarity and entropy of feature maps, Neurocomputing 544 (2023) 126297.
[40] Y. Zniyed, T.P. Nguyen, et al., Efficient tensor decomposition-based filter pruning, Neural Netw. 178 (2024) 106393.
[41] M.K. Lee, S. Lee, S.H. Lee, B.C. Song, Channel pruning via gradient of mutual information for light-weight convolutional neural networks, in: 2020 IEEE International Conference on Image Processing (ICIP), IEEE, 2020, pp. 1751–1755.
[42] S. Lin, R. Ji, Y. Li, C. Deng, X. Li, Toward compact convnets via structure-sparsity regularized filter pruning, IEEE Trans. Neural Netw. Learn. Syst. 31 (2) (2019) 574–588.
[43] Y. Zhou, Y. Zhang, Y. Wang, Q. Tian, Accelerate CNN via recursive Bayesian pruning, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 3306–3315.
[44] Y. Liu, D. Wu, W. Zhou, K. Fan, Z. Zhou, Eacp: an effective automatic channel pruning for neural networks, Neurocomputing 526 (2023) 131–142.
[45] Y. Xue, W. Yao, S. Peng, S. Yao, Automatic filter pruning algorithm for image classification, Appl. Intell. 54 (1) (2024) 216–230.